CRM Sales Pipeline Cleaning & Performance Dashboard

Data cleaning, relational joins, Python preprocessing, validation, analysis, and executive reporting

8,800	$10.01M	63.2%	4,238
Opportunities	Won Revenue	Closed-Deal Win Rate	Won Deals

Role: Data Analyst **Tools:** Python, Excel / Google Sheets **Dataset:** Maven Analytics CRM Sales Opportunities (public practice dataset)

This PDF combines the finished workbook and the preprocessing script into one portfolio artifact. It shows the analytical deliverable first, then documents the reproducible Python workflow used to create the analysis-ready master dataset.

1. Project Overview

The project starts with four related CRM tables - sales pipeline, customer accounts, products, and sales teams. The workflow preserves the raw files, audits data quality, standardizes known inconsistencies, joins the tables into a single opportunity-level dataset, reconciles source totals, and then analyzes sales performance across reps, teams, products, sectors, accounts, and time.

Area	What was done
Data audit	Validated 8,800 unique opportunity IDs, reviewed stage-dependent blanks, and checked cross-table keys.
Standardization	Mapped GTXPro to GTX Pro and corrected technolgy to technology only in derived data.
Relational joins	Enriched opportunities with manager, region, product series, list price, sector, account revenue, employees, and office location.
Calculated fields	Added sales-cycle days, won/lost/open flags, close month/quarter/year, and price variance metrics.
Validation	Reconciled the master dataset to 8,800 opportunities and $10,005,534 in won revenue with no duplicate opportunity IDs.
Reporting	Built management views for pipeline KPIs, reps, teams, products, sectors, accounts, and monthly performance.

Key business findings: GTX Pro and GTX Plus Pro generated about 61.4% of won revenue from only 27.8% of opportunities. Melvin Marxen's team generated the most total won revenue, while Rocco Neubert's team led revenue per opportunity. Darcel Schlecht was the highest-revenue rep, and Retail was the highest-revenue sector.

2. Executive Dashboard

The dashboard combines company-level KPIs, major findings, and visual summaries of monthly, product, manager, and rep performance.

CRM SALES PIPELINE & REVENUE PERFORMANCE

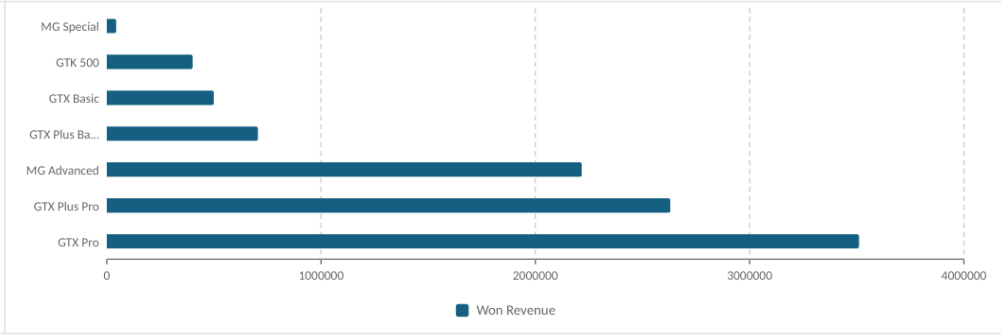
Relational CRM analysis • Python preprocessing • Excel reporting & management dashboard

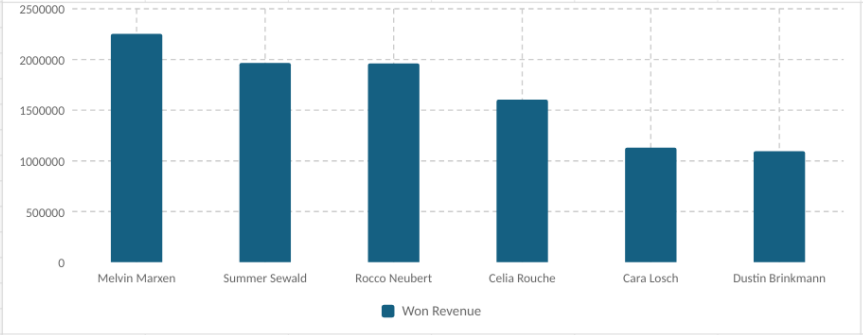
WON REVENUE	CLOSED-DEAL WIN RATE	WON DEALS	AVG WON DEAL
$10,005,534	63.2%	4238	$2,361

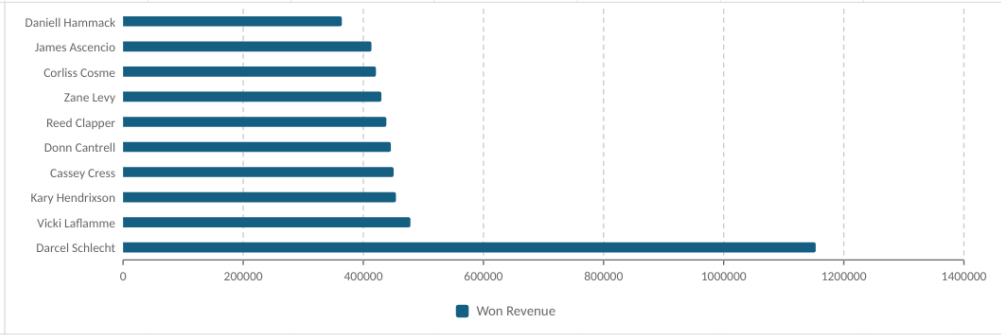
KEY FINDINGS

- Overall performance: $10,005,534 in won revenue from 4,238 wins; closed-deal win rate 63.2%.
- GTX Pro + GTX Plus Pro produced 61.4% of won revenue from only 27.8% of opportunities.
- Melvin Marxen's team generated the most revenue ($2,251,930); Rocco Neubert's team led revenue per opportunity ($1,477).
- Darcel Schlecht was the top rep at $1,153,214, or 11.5% of company won revenue.
- Retail led sectors at $1,867,528; Kan-code was the highest-revenue named account at $341,455.









Source: Maven Analytics CRM Sales Opportunities public practice dataset. Raw source files are preserved separately; standardized fields and joins were produced in a reproducible Python preprocessing workflow.

3. Data Quality Audit

The audit distinguishes real data-quality issues from structurally valid blanks and documents reconciliation to the source pipeline.

	A	B	C	D
1	**DATA QUALITY AUDIT & RECONCILIATION**			
2				
3				
4	**Audit Check**	**Source**	**Master**	**Status / Note**
5	Pipeline records	8800	8800	PASS
6	Unique opportunity IDs	8800	8800	PASS – no duplicates
7	Missing accounts	1425	1425	Expected / preserved
8	Missing engage dates	500	500	Matches Prospecting
9	Missing close dates	2089	2089	Matches open deals
10	Missing close values	2089	2089	Matches open deals
11	Product lookup failures before standardization	1480	0	GTXPro → GTX Pro
12	Sales-agent lookup failures	0	0	PASS
13	Populated account lookup failures	0	0	PASS
14	Won revenue	10005534	10005534	PASS – fully reconciled

4. Product Performance

Product analysis compares opportunity volume, win rate, won revenue, average deal value, and pricing variance.

	A	B	C	D	E	F	G	H	I	J	K
1	PRODUCT PERFORMANCE & PRICING										
2											
3											
4	Product	Series	List Price	Opportunities	Won	Lost	Open	Win Rate	Won Revenue	Avg Won Deal	Avg Price Variance %
5	GTX Pro	GTX	$4,821.00	1480	729	418	333	63.6%	$3,510,578.00	$4,815.61	-0.1%
6	GTX Plus Pro	GTX	$5,482.00	968	479	266	223	64.3%	$2,629,651.00	$5,489.88	0.1%
7	MG Advanced	MG	$3,393.00	1412	654	430	328	60.3%	$2,216,387.00	$3,388.97	-0.1%
8	GTX Plus Basic	GTX	$1,096.00	1383	653	398	332	62.1%	$705,275.00	$1,080.05	-1.5%
9	GTX Basic	GTX	$550.00	1866	915	521	430	63.7%	$499,263.00	$545.64	-0.8%
10	GTK 500	GTK	$26,768.00	40	15	10	15	60.0%	$400,612.00	$26,707.47	-0.2%
11	MG Special	MG	$55.00	1651	793	430	428	64.8%	$43,768.00	$55.19	0.4%

5. Sales Rep Performance

Rep analysis separates raw volume from effectiveness using revenue, win rate, average deal size, sales cycle, and revenue per opportunity.

	A	B	C	D	E	F	G	H	I	J	K	L	M
1	SALES REP PERFORMANCE												
2													
3													
4	Sales Agent	Manager	Region	Opportunities	Won	Lost	Open	Win Rate	Won Revenue	Avg Won Deal	Avg Sales Cycle	Revenue / Opportun	Win Rate vs Compan
5	Darcel Schlecht	Melvin Marxen	Central	747	349	204	194	63.1%	$1,153,214.00	$3,304.34	49.4	$1,543.79	0.0%
6	Vicki Laflamme	Celia Rouche	West	451	221	126	104	63.7%	$478,396.00	$2,164.69	53.6	$1,060.75	0.5%
7	Kary Hendrixson	Summer Sewald	West	438	209	126	103	62.4%	$454,298.00	$2,173.67	53.0	$1,037.21	-0.8%
8	Cassey Cress	Rocco Neubert	East	346	163	98	85	62.5%	$450,489.00	$2,763.74	48.8	$1,301.99	-0.7%
9	Donn Cantrell	Rocco Neubert	East	275	158	117	0	57.5%	$445,860.00	$2,821.90	53.3	$1,621.31	-5.7%
10	Reed Clapper	Rocco Neubert	East	237	155	82	0	65.4%	$438,336.00	$2,827.97	49.0	$1,849.52	2.3%
11	Zane Levy	Summer Sewald	West	349	161	100	88	61.7%	$430,068.00	$2,671.23	46.9	$1,232.29	-1.5%
12	Corliss Cosme	Cara Losch	East	310	150	79	81	65.5%	$421,036.00	$2,806.91	51.1	$1,358.18	2.4%
13	James Ascencio	Summer Sewald	West	267	135	71	61	65.5%	$413,533.00	$3,063.21	53.4	$1,548.81	2.4%
14	Daniell Hammack	Rocco Neubert	East	259	114	73	72	61.0%	$364,229.00	$3,194.99	46.6	$1,406.29	-2.2%
15	Maureen Marcano	Summer Sewald	West	285	149	64	72	70.0%	$350,395.00	$2,351.64	54.1	$1,229.46	6.8%
16	Gladys Colclough	Melvin Marxen	Central	317	135	97	85	58.2%	$345,674.00	$2,560.55	53.0	$1,090.45	-5.0%

6. Monthly Performance

Monthly analysis tracks closed deals, wins, losses, win rate, won revenue, and average won sales cycle.

	A	B	C	D	E	F	G
1	MONTHLY SALES PERFORMANCE						
2							
3							
4	Month	Closed Deals	Won	Lost	Win Rate	Won Revenue	Avg Won Sales Cycle
5	2017-03	647	531	116	82.1%	$1,134,672.00	65.6
6	2017-04	586	285	301	48.6%	$721,932.00	6.1
7	2017-05	805	438	367	54.4%	$1,025,713.00	61.6
8	2017-06	641	531	110	82.8%	$1,338,466.00	66.4
9	2017-07	627	308	319	49.1%	$696,932.00	6.4
10	2017-08	785	446	339	56.8%	$1,050,059.00	61.7
11	2017-09	635	503	132	79.2%	$1,235,264.00	63.4
12	2017-10	566	279	287	49.3%	$731,980.00	6.2
13	2017-11	768	406	362	52.9%	$938,943.00	57.1
14	2017-12	651	511	140	78.5%	$1,131,573.00	67.3

7. Python Preprocessing Workflow

The accompanying Python script reproduces the preprocessing layer used for the workbook. It loads the four source CSVs, creates lookup maps, standardizes known source inconsistencies, performs relational enrichment, creates calculated analysis fields, runs reconciliation assertions, and outputs **master_opportunities.csv**.

Step	Operation	Purpose
1	**Load raw CSVs**	accounts.csv, products.csv, sales_pipeline.csv, sales_teams.csv
2	**Build lookup maps**	Account, product, and sales-agent keys
3	**Standardize**	GTXPro -> GTX Pro; technolgy -> technology in the derived layer
4	**Join / enrich**	Manager, region, product attributes, and customer attributes
5	**Calculate**	Sales cycle, stage flags, time fields, and price variance
6	**Validate**	Assert row count, ID uniqueness, and won-revenue reconciliation
7	**Export**	Create master_opportunities.csv

The validation assertions are important because they make the script fail loudly if preprocessing changes the number of records, creates duplicate opportunity IDs, or changes total won revenue.

Appendix A - Python Preprocessing Script (1/3)

Full script included so the preprocessing and validation workflow can be reviewed directly.

```
"""
CRM Sales Opportunities preprocessing workflow.

Inputs:
- accounts.csv
- products.csv
- sales_pipeline.csv
- sales_teams.csv

Outputs:
- master_opportunities.csv

This script preserves raw source files, standardizes known source inconsistencies
only in derived data, enriches opportunities with lookup fields, creates calculated
analysis fields, and reconciles source totals after processing.
"""

import csv
from datetime import datetime
from pathlib import Path

BASE = Path(".")

def read_csv(name):
    with open(BASE / name, newline="", encoding="utf-8-sig") as f:
        return list(csv.DictReader(f))

accounts = read_csv("accounts.csv")
products = read_csv("products.csv")
pipeline = read_csv("sales_pipeline.csv")
teams = read_csv("sales_teams.csv")

account_map = {r["account"]: r for r in accounts}
product_map = {r["product"]: r for r in products}
team_map = {r["sales_agent"]: r for r in teams}

def parse_date(value):
    return datetime.strptime(value, "%Y-%m-%d").date() if value else None

master = []

for row in pipeline:
    product = "GTX Pro" if row["product"] == "GTXPro" else row["product"]

    product_record = product_map.get(product, {})
    account_record = account_map.get(row["account"], {}) if row["account"] else {}
    team_record = team_map.get(row["sales_agent"], {})

    sector_raw = account_record.get("sector", "")
    sector = "technology" if sector_raw == "technolgy" else sector_raw

    engage_date = parse_date(row["engage_date"])
    close_date = parse_date(row["close_date"])
    close_value = float(row["close_value"]) if row["close_value"] else None
    list_price = (
        float(product_record["sales_price"])
        if product_record.get("sales_price")
        else None
    )
```

Appendix A - Python Preprocessing Script (2/3)

Full script included so the preprocessing and validation workflow can be reviewed directly.

```
    stage = row["deal_stage"]
    sales_cycle_days = (
        (close_date - engage_date).days
        if engage_date and close_date
        else None
    )

    price_variance = (
        close_value - list_price
        if stage == "Won"
        and close_value is not None
        and list_price is not None
        else None
    )

    master.append({
        "opportunity_id": row["opportunity_id"],
        "sales_agent": row["sales_agent"],
        "manager": team_record.get("manager", ""),
        "regional_office": team_record.get("regional_office", ""),
        "product_raw": row["product"],
        "product": product,
        "product_series": product_record.get("series", ""),
        "list_price": list_price,
        "account": row["account"],
        "sector_raw": sector_raw,
        "sector": sector,
        "account_revenue": (
            float(account_record["revenue"])
            if account_record.get("revenue")
            else None
        ),
        "employees": (
            int(account_record["employees"])
            if account_record.get("employees")
            else None
        ),
        "office_location": account_record.get("office_location", ""),
        "deal_stage": stage,
        "engage_date": engage_date,
        "close_date": close_date,
        "close_value": close_value,
        "sales_cycle_days": sales_cycle_days,
        "won_flag": int(stage == "Won"),
        "lost_flag": int(stage == "Lost"),
        "open_flag": int(stage in {"Prospecting", "Engaging"}),
        "close_month": close_date.strftime("%Y-%m") if close_date else "",
        "close_quarter": (
            f"{close_date.year}-Q{((close_date.month - 1) // 3) + 1}"
            if close_date else ""
        ),
        "close_year": close_date.year if close_date else None,
        "price_variance": price_variance,
        "price_variance_pct": (
            price_variance / list_price
            if price_variance is not None and list_price
            else None
        ),
    })
```

Appendix A - Python Preprocessing Script (3/3)

Full script included so the preprocessing and validation workflow can be reviewed directly.

```
# Reconciliation checks
assert len(master) == len(pipeline)
assert len({r["opportunity_id"] for r in master}) == len(master)

source_won_revenue = sum(
    float(r["close_value"])
    for r in pipeline
    if r["deal_stage"] == "Won" and r["close_value"]
)
master_won_revenue = sum(
    r["close_value"]
    for r in master
    if r["deal_stage"] == "Won"
)

assert source_won_revenue == master_won_revenue

headers = list(master[0].keys())
with open("master_opportunities.csv", "w", newline="", encoding="utf-8") as f:
    writer = csv.DictWriter(f, fieldnames=headers)
    writer.writeheader()
    writer.writerows(master)

print(f"Created master_opportunities.csv with {len(master):,} rows")
print(f"Won revenue reconciled: ${master_won_revenue:,.0f}")
```

Portfolio Notes

Project type: Independent portfolio project using a public practice dataset; not client data.

Dataset source: Maven Analytics CRM Sales Opportunities - https://mavenanalytics.io/data-playground/crm-sales-opportunities

Deliverables represented in this PDF: the completed analytical workbook and the Python preprocessing script. The original Excel workbook and .py file can also be provided separately if a client requests editable source files.

Suggested portfolio title: CRM Sales Pipeline Cleaning & Performance Dashboard